

Name: Adalynn Cadence Everly

ID: 1001852753

Class: CSE-4321-001

HW 7:

Exercises Section 9.2: Problems 1, and 3.

HW7

1. Provide reachability conditions, infection conditions, propagation conditions, and test case values to kill mutants 2, 4, 5, and 6 in Figure 9.1.

Original Method	With Embedded Mutants
<pre>int Min (int A, int B) { int minVal; minVal = A; if (B < A) { minVal = B; } return (minVal); } // end Min</pre>	<pre>int Min (int A, int B) { int minVal; minVal = A; Δ1 minVal = B; if (B < A) Δ2 if (B > A) Δ3 if (B < minVal) { minVal = B; Δ4 Bomb(); minVal = A; minVal = failOnZero (B); } return (minVal); } // end Min</pre>

Figure 9.1. Method Min and six mutants.

Q1

Conditions

HW7

1) Reachability 4) Test case val: 2, 4, 5, & 6 Mutant = Mut.

2) Expectation

3) Propagation

Mut 2: If ($B > A$) instead of if ($B < A$)

- 1) $B & A$ must be different
- 2) The condition $B > A$ must evaluate true (When $B < A$) so wrong branch when executed.
- 3) The return val. must be affected. Will/must observe wrong output
- 4) $\text{Min}(3, 5) \rightarrow$ original returns 3. Mut. returns 5 (Difference shows the fault)

Mut 4: $\text{MinVal} = B$; becomes $\text{Bomb}()$;

- 1) $B < \text{MinVal}$ must be true
- 2) The statement is executed i.e. the block is entered
- 3) $\text{Bomb}()$ must cause observable behavior. Crashes or behaves differently
- 4) $\text{Min}(3, 2) \rightarrow$ original returns 2, Mut. may crash or behave differently.

Mut. 5: $\text{minVal} = B$; becomes $\text{minVal} = A$;

- 1) $B < \text{minVal}$ must be true.
- 2) The assignment to $\text{MinVal} = A$ must be executed.
- 3) The fin. val. returned must reflect it, Not B.
- 4) $\text{Min}(5, 2) \rightarrow$ original returns 2. Mut returns 5 $\rightarrow$ clearly wrong.

Mut 6: $\text{minVal} = B$; becomes $\text{minVal} = \text{Fail} / \text{OnZero}$

- 1) $B < \text{minVal}$ must be true.
- 2) The mutation executes
- 3) Propagation: If $B == 0$, mutation causes failure or crash
- 4) $\text{Min}(3, 6) \rightarrow$ original returns 3, Mutation may throw of fail if $\text{failOnZero}()$ is defined or crashes.

3. Answer questions (a) through (d) for the mutant on line 6 in the method sum().

```

/**
 * Sum values in an array
 *
 * @param x array to sum
 *
 * @return sum of values in x
 * @throws NullPointerException if x is null
 */
1. public static int sum(int[] x)
2. {
3.     int s = 0;
4.     for (int i=0; i < x.length; i++) {
5.         {
6.             s = s + x[i];
6'.      // s = s - x[i]; //AOR
7.         }
8.     return s;
9. }

```

(a) If possible, find test inputs that do **not** reach the mutant.

Q2 HW 7 P 2,

Line 6: $s = s + x[i];$ → mutant $s = s - x[i];$ TC = test case
 Input that doesn't enter the loop
 • TC: sum (new int[0] empty array)
 • Loop never runs, line 6 is never executed.
 • We reach 6, but both versions yield the same

(b) If possible, find test inputs that satisfy reachability but **not infection** for the mutant.

• TC: sum (new int[0] empty array)
 • Loop never runs, line 6 is never executed.
 • We reach 6, but both versions yield the same value
 T.C: sum (new int[1] {0})
 $s = s + 0$ VS $s = s - 0 \Rightarrow$ results are both $s = 0$.
 • Internal state changes, but end result = same

(c) If possible, find test inputs that satisfy reachability and infection, but **not propagation** for the mutant.

Q) internal state changes, but end result = same
 T.C: sum (new int [] { 1, -13 })
 original: $0 + 1 + (-13) = -12$
 mutant: $0 - 1 - (-13) = 12$
 Different intermediate values, same final output.
 ∴ Final result is different, & visible.

(d) If possible, find test inputs that strongly kill the mutants.

Q) Final result is different, & visible.
 T.C: sum (new int [] { 1, 23 })
 original: $0 + 1 + 23 = 24$
 mutant: $0 - 1 - 23 = -24$ ← This is very wrong